Bagging: Bootstrap Aggregation

Contents

1. Overview . . . 2
2. The core idea . . . 2
   - 2.1 Bootstrapping . . . 3
   - 2.2 Aggregation . . . 4
   - 2.3 Drawing with replacement . . . 4
   - 2.4 Where bootstrapping comes from . . . 6
3. Why bagging works . . . 7
   - 3.1 The bias-variance problem . . . 7
   - 3.2 How bagging lowers variance . . . 8
4. When to use bagging . . . 9
5. Bagging by hand . . . 10
   - 5.1 The data . . . 10
   - 5.2 Bootstrapping: three samples, three trees . . . 10
   - 5.3 Aggregation: the vote . . . 11
6. Types of bagging . . . 12
   - 6.1 Bagging . . . 12
   - 6.2 Pasting . . . 12
   - 6.3 Random subspaces . . . 13
   - 6.4 Random patches . . . 13
7. Summary . . . 14
8. Sources . . . 14
9. Key terms . . . 15

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Overfitting; Expected value and variance of a random variable; Bias-variance trade-off; Decision trees; Grid and random search; Ensemble learning.
- **Leads to:** Random forest; OOB score.
- **Compare with:** Voting ensembles; Random forest; Boosting.

1. Overview

Key point: Bagging trains many copies of one algorithm, each on a different random sample of the observations (bootstrapping), and combines their predictions by vote or mean (aggregation). Bagging keeps the low bias of a flexible model and cuts its **variance** (G-2073).

Bagging, short for **bootstrap aggregation**, is one of the two most important ensemble techniques, with boosting (see the four types of ensemble). The name joins its two steps: **B**ootstrapping and **AGG**regation.

This Note covers the core idea, why bagging works, when to use it, a small worked example in code, and the four ways to sample: bagging, pasting, random subspaces and random patches. The next two Notes apply it to classification (bagging classifier) and regression (**bagging regressor** (G-252)).

The Notebook (`ML-099-bagging-intuition.ipynb`) runs the worked example.

2. The core idea

Key point: Same algorithm for every base model; different random observations for each; then vote or average.

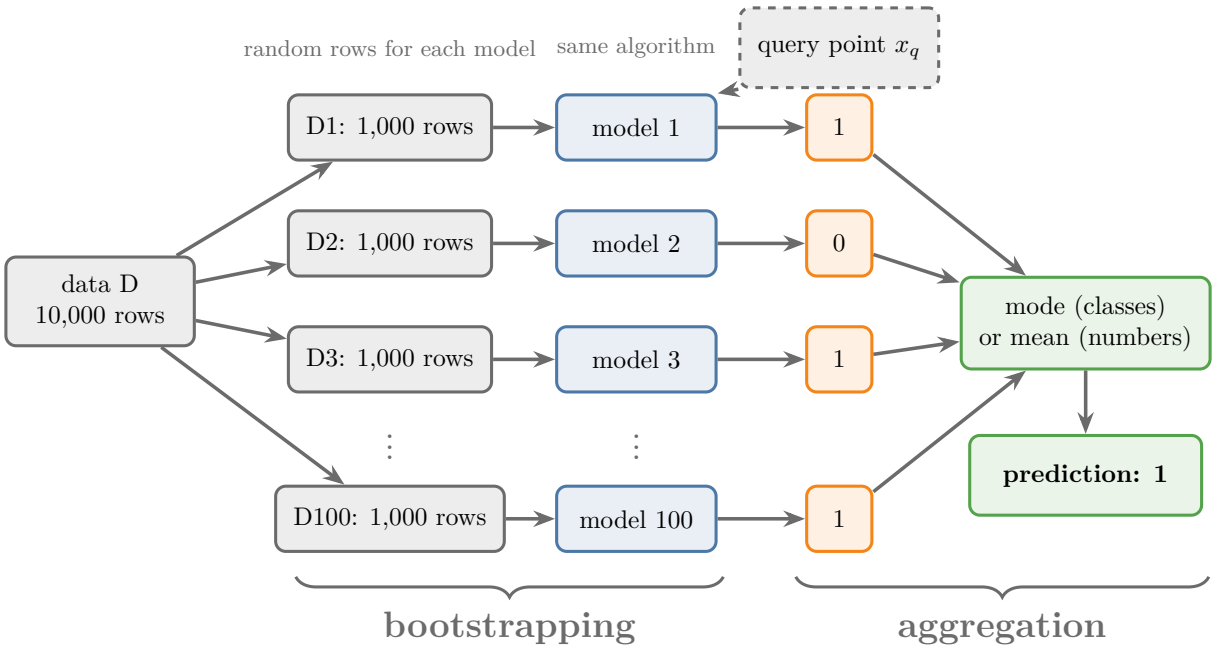


Figure 1: Bagging: each base model trains on its own random sample of the observations; the predictions are combined by mode or mean

Take a classification dataset D of 10,000 **observations** (records; each is one row of the data table) with a yes/no **target** (the output we predict). Figure 1 shows the two steps.

2.1 Bootstrapping

> **Key point:** Draw a random sample of observations for each base model and train it on that sample.

1. Choose a number of base models, say 100 (it could be 10 or 1,000). They all use the **same algorithm**: all decision trees, all KNN, or all SVMs. Unlike voting, bagging does not mix algorithms: by definition it makes many versions of one predictor (Breiman, 1996, section 1).
2. Choose how many observations each model sees, say 1,000.
3. Draw 1,000 observations at random from D: this is dataset D1. Train model 1 on D1.
4. Draw another random 1,000 observations: D2, for model 2. D2 differs from D1, because the observations are random. And so on, up to D100.

In bagging, drawing these random samples is called **bootstrapping**, after the statistics technique of resampling a dataset with replacement (section 2.3; ESL §8.7). Each base model is trained on a different subset, so each learns something slightly different, and the ensemble gets its variety from the data rather than from the algorithms.

2.2 Aggregation

> **Key point:** Send the query point to every model and combine their answers: the mode for classification, the mean for regression.

For a new query point x_q, every trained model predicts, and the answers are combined as in every ensemble (see how an ensemble predicts): the most common answer, the **mode**, for classification (in Figure 1, most models say 1, so the answer is 1) and the mean for regression. This combining step is the **aggregation**.

So the only difference from voting (the core idea of voting) is where the variety comes from:

	Voting	Bagging
Base models	different algorithms	one algorithm
Data each model sees	the same data	a different random sample

Because the base models differ, the probability argument for voting applies, and the combined model usually performs better than its base models. In practice scikit-learn does all of this for us; BaggingClassifier in code shows how.

2.3 Drawing with replacement

> **Key point:** In a bootstrap sample an observation can be drawn more than once, and some are never drawn: on average a sample holds about 63% of the distinct observations.

Observations can be drawn **with replacement**: after an observation is drawn, it is put back and can be drawn again. Picture a bag of numbered balls: we pull one out, write down its number, and drop it back in before the next pull. Figure 2 draws 10 observations from 10, with replacement, twice.

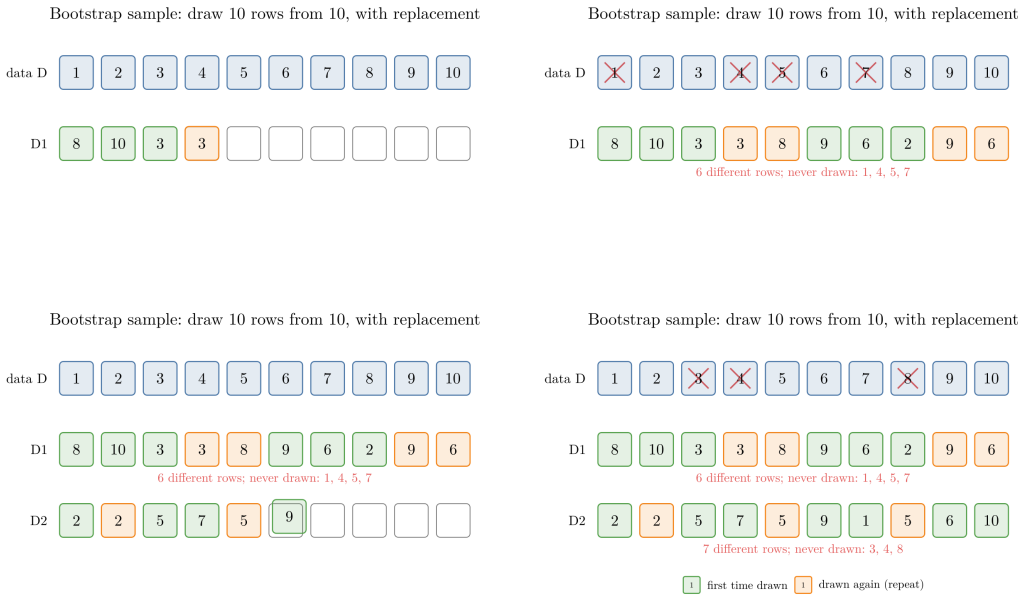


Figure 2: Drawing two bootstrap samples of 10 observations from 10: green rows are drawn for the first time, orange rows are repeats, crossed rows are never drawn

- D1 holds only 6 different observations: 3, 6, 8 and 9 appear twice, and observations 1, 4, 5 and 7 are never drawn.
- D2 holds 7 different observations: 2 appears twice and 5 three times, while 3, 4 and 8 are missing.

A sample of the same size as the data, drawn with replacement, is a **bootstrap sample** (Efron and Tibshirani, 1993; checking multicollinearity refitted a model on such resamples). Drawing observations without replacement is also possible; section 6 calls that pasting.

Extra: How many different observations does a bootstrap sample hold?

1. **In words:** one draw misses a given observation with probability $1 - 1/n$. All n draws miss it with probability $(1 - 1/n)^n$. So the share of observations that appear at least once is $1 - (1 - 1/n)^n$.
2. **Formula:** as n grows, $(1 - 1/n)^n$ approaches $1/e$, so

$$\text{share} = 1 - \left(1 - \frac{1}{n}\right)^n$$

$$\text{share} \longrightarrow 1 - \frac{1}{e} \approx 0.632$$

3. **Example:** for $n = 10$:

$$(0.9)^{10} = 0.349$$

$$1 - 0.349 = 0.651$$

That is about 6.5 distinct observations out of 10 (Figure 2 got 6 and 7). For $n = 10{,}000$ the share is 0.632.

The Notebook checks this by simulation: 0.655 for $n = 10$, 0.632 for $n = 1{,}000$ and above. The roughly 37% of observations a model never sees are its **out-of-bag** observations. The **OOB score** (out-of-bag score) uses them to score the model: each observation is predicted only by the models that never saw it, and the share of correct answers is the score (OOB score).

2.4 Where bootstrapping comes from

Key point: Bootstrapping was invented to answer one question: how much would a result change if we collected the data again? Each bootstrap sample stands in for one fresh dataset.

Bootstrapping is older than bagging, and its first use explains why the samples are drawn with replacement and have the same size as the data.

The problem. Suppose 8 patients take a new drug, and we measure how each one responds: 5 feel better (positive values) and 3 feel worse (negative values). The mean response is 0.5 (example data). Is the drug useful, or would 8 other patients give a mean near 0? Repeating the experiment many times would tell us, but each repeat costs time and money.

The idea. We treat the 8 values we have as a stand-in for all patients, and make new datasets from them. Figure 3 shows the steps.

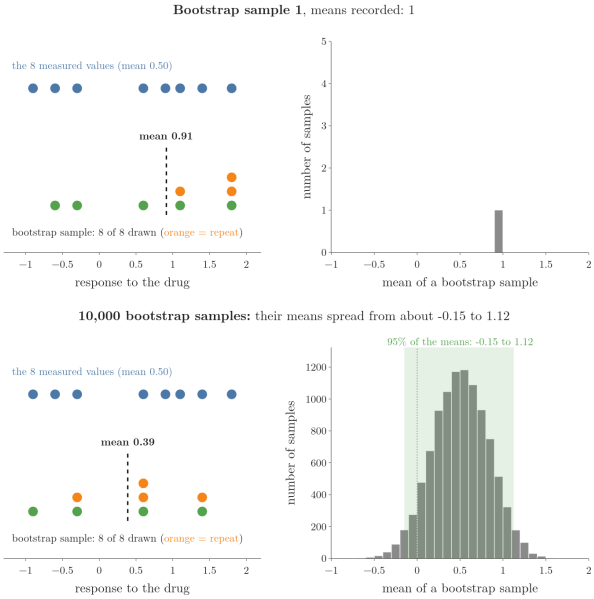


Figure 3: Bootstrapping the mean of 8 drug responses (example data). Left: the 8 measured values (blue) and one bootstrap sample drawn from them (green: first draw of a value; orange: a repeat). Right: the means of the bootstrap samples so far. Idea after StatQuest, "Bootstrapping Main Ideas!!!".

1. **Draw a bootstrap sample.** Pick one of the 8 values at random, copy it, and put it back. Do this 8 times. The first sample is -0.6, -0.3, 0.6, 1.1, 1.1, 1.8, 1.8, 1.8: two values came up more than once, and three were never drawn.
2. **Calculate.** The mean of this sample is 0.91, not 0.5, because the sample differs from the original data.
3. **Record** the mean in a histogram (a chart whose bars count how many values fall in each range).
4. **Repeat** steps 1 to 3 many times; Figure 3 goes up to 10,000 samples.

What the histogram says. The 10,000 means show how the mean could vary if we redid the experiment. Their standard deviation (how far they typically lie from their own mean), 0.32, estimates the **standard error** (G-1872) of the mean. The middle 95 percent of them lie between -0.15 and 1.12. This interval contains 0, so with only 8 patients we cannot rule out that the drug does nothing.

The two design choices now have a reason:

- **With replacement:** without it, 8 draws from 8 values would return the same 8 values every time, and every sample would be identical.
- **Same size as the data:** the spread of a mean depends on how many values go into it, so each sample must be as large as the real experiment.

The method works for any statistic, not only the mean: in step 2 we could calculate the median or the standard deviation instead.

The link to bagging. Bagging keeps steps 1 and 4 and changes step 2: instead of calculating a mean on each bootstrap sample, it trains a model on it. Each model stands for "what we would have learned from another dataset", and section 3 shows why averaging them helps.

3. Why bagging works

> **Key point:** Bagging takes low-bias, high-variance models, such as fully grown trees, and averages away much of their **variance** (G-2073), leaving a model with low bias and low variance.

3.1 The bias-variance problem

> **Key point:** We want low bias and low variance, but in a single model reducing one usually raises the other.

From **bias** (G-287) and **variance** (G-2073): **bias** is a model's inability to fit even its training data, and **variance** is how much its predictions change when the training data changes. A good model has low bias (accurate on the training data) and low variance (consistent results when the data changes slightly).

The trouble is that the two pull against each other. Most algorithms end up either low bias, high variance (they overfit) or high bias, low variance (they underfit).

3.2 How bagging lowers variance

> **Key point:** A change in the data is spread across many models, so no single change can swing the ensemble.

Bagging uses base models that are **low bias, high variance**:

- a **fully grown decision tree** (`max_depth=None`, see **max_depth** (G-1184)), which fits its training data perfectly but overfits;
- other **unstable** models, whose fit changes a lot when the data changes a little: Breiman (1996, section 1) found trees and neural networks unstable, and nearest-neighbour methods stable.

Their bias is already low, and averaging leaves it about where it was; what averaging cuts is the **variance** (G-2073) (ESL §8.7, §15.2; Breiman, 1996, section 4).

Suppose we replace 100 of the 10,000 observations with very different, noisy ones. A single tree trained on all the data would change its logic completely because of those 100 observations. In bagging, each tree sees its own random 1,000 observations, so the 100 new ones are spread out: one tree gets 12 of them, another 5, another none, another 9.

No single tree absorbs all the change, so the behaviour of each tree changes less, and the average changes even less. The ensemble gives consistent results: **low variance**. Since its base models were already accurate (low **bias** (G-287)), we end up with the combination we wanted: **low bias, low variance**.

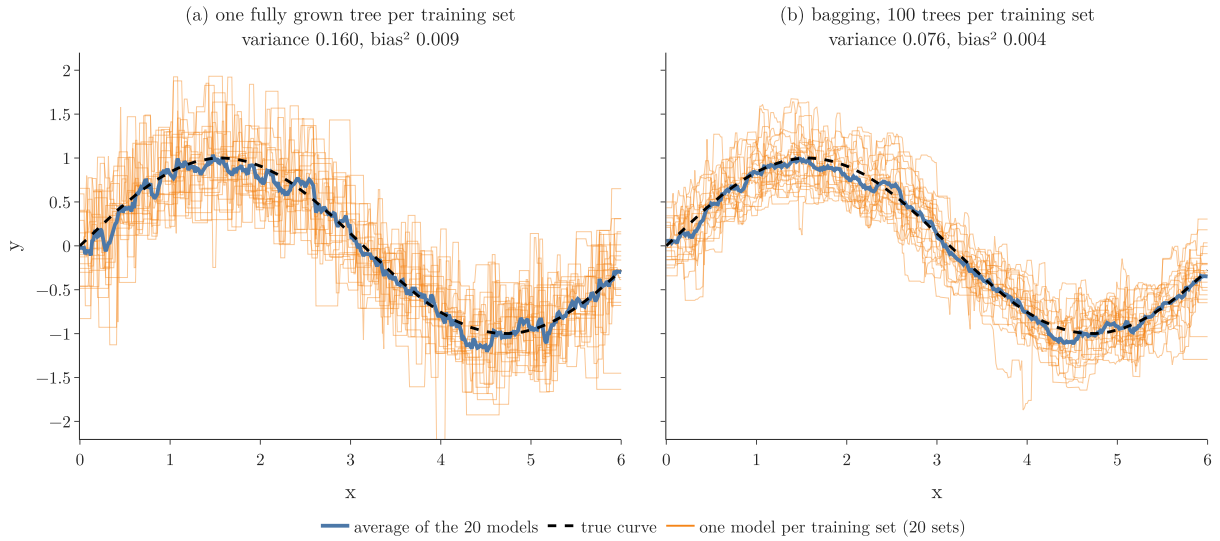


Figure 4: The same experiment repeated on 20 different training sets: (a) a single fully grown tree, (b) bagging with 100 trees. The spread of the orange curves is the variance

Figure 4 shows this. We draw 20 different training sets of 60 noisy points from a sine curve and fit a model to each:

- **(a) one fully grown tree per set:** the 20 curves swing wildly around the truth. Average variance **0.160**.
- **(b) bagging with 100 trees per set:** the 20 curves stay close together. Average variance **0.076**, less than half.

The squared bias stays tiny in both (0.009 and 0.004): both average curves follow the true curve.

4. When to use bagging

> **Key point:** Whenever the model is low bias and high **variance** (G-2073) (it overfits), try bagging; it is not limited to decision trees.

Bagging is worth trying whenever a model overfits: low **bias** (G-287), high variance. Breiman (1996, section 1) calls such models **unstable**: a small change in the training data causes a large change in the model. Bagging helps unstable models and can slightly hurt stable ones, such as nearest-neighbour methods (Breiman, 1996, sections 1 and 6.3). Bagging is also what makes **random forests** (bagging with decision trees, see how a random forest works) so popular.

Figure 5 measures the difference on the setup of Figure 4: the same 20 training sets, once with a fully grown tree (unstable) and once with a **5-nearest-neighbour** model (G-998), which averages the 5 closest training points and is stable.

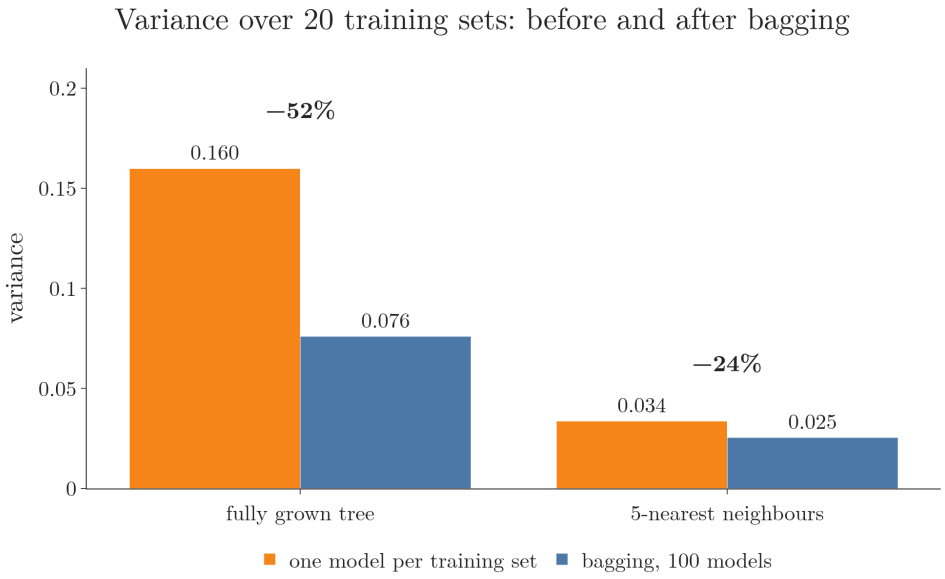


Figure 5: Variance over the 20 training sets of Figure 4, for one model per set (orange) and for bagging with 100 models per set (blue).

- **Fully grown tree.** One tree per set has variance 0.160; bagging cuts it to 0.076, a 52 percent cut.

- **5-nearest neighbours.** One model per set has variance 0.034, already low because each prediction averages 5 points; bagging only brings it to 0.025, a 24 percent cut.

The more a model changes with its data, the more there is for bagging to average away.

A common misunderstanding is that bagging only works with decision trees. Trees are the usual choice because they are unstable, so they gain the most. **Any algorithm** can be bagged, but the gain depends on how unstable it is: the bagging classifier bags KNN and SVMs (support vector machines) too, and they gain little or nothing (see other base models).

5. Bagging by hand

> **Key point:** Three trees, each trained on 8 observations drawn with replacement, then a majority vote.

To see each step, we bag three decision trees on a tiny dataset.

5.1 The data

> **Key point:** Versicolor against virginica, with sepal width and petal length; 10 training observations.

From the iris data (see the data of softmax regression) we keep versicolor (class 1) and virginica (class 2), two species that overlap, and two **features** (input variables, columns of the data table): sepal width and petal length. That leaves 100 observations.

The rows of the table are stored species by species, so we first shuffle them with `df.sample(100)`. Then we take tiny sets so that every step can be printed: 10 random observations for training, 5 for validation and 5 for testing.

5.2 Bootstrapping: three samples, three trees

> **Key point:** Each tree gets its own 8 observations drawn with replacement, and grows fully on them.

> **Python:** One bootstrap sample and its tree.
>
> ```
> # 8 rows from the 10, with replacement
> df_bag = df_train.sample(8, replace=True)
> X, y = df_bag.iloc[:, 0:2], df_bag.iloc[:, -1]
> # max_depth not set: a fully grown tree
> tree = DecisionTreeClassifier().fit(X, y)
> ```
>
> `sample(n)` draws `n` random rows; `replace=True` puts each row back after drawing it, so rows can repeat. The default, `replace=False`, never repeats a row.

Running this three times gives three different samples. In the Notebook, the first sample holds row 91 three times and row 69 twice; the third holds row 105 three times. With so few observations, each tree makes a single split on petal length, but each at a different place:

Tree	Split	Validation accuracy (5 observations)
1	petal length $\leq 5.20 \rightarrow$ class 1	0.4
2	petal length $\leq 4.90 \rightarrow$ class 1	0.8
3	petal length $\leq 4.95 \rightarrow$ class 1	0.6

5.3 Aggregation: the vote

> **Key point:** For a virginica with sepal width 2.2 and petal length 5.0, the trees say 1, 2, 2; the majority says 2, which is right.

We send a new flower to all three trees: sepal width 2.2, petal length 5.0. The flower is a real virginica (class 2) from the iris data, and it is not one of the 10 training flowers, so no tree has seen it.

- Tree 1 (split at 5.20): 5.0 is below, so **class 1**.
- Tree 2 (split at 4.90): above, so **class 2**.
- Tree 3 (split at 4.95): above, so **class 2**.

Figure 6 draws the three trees on the 10 training flowers.

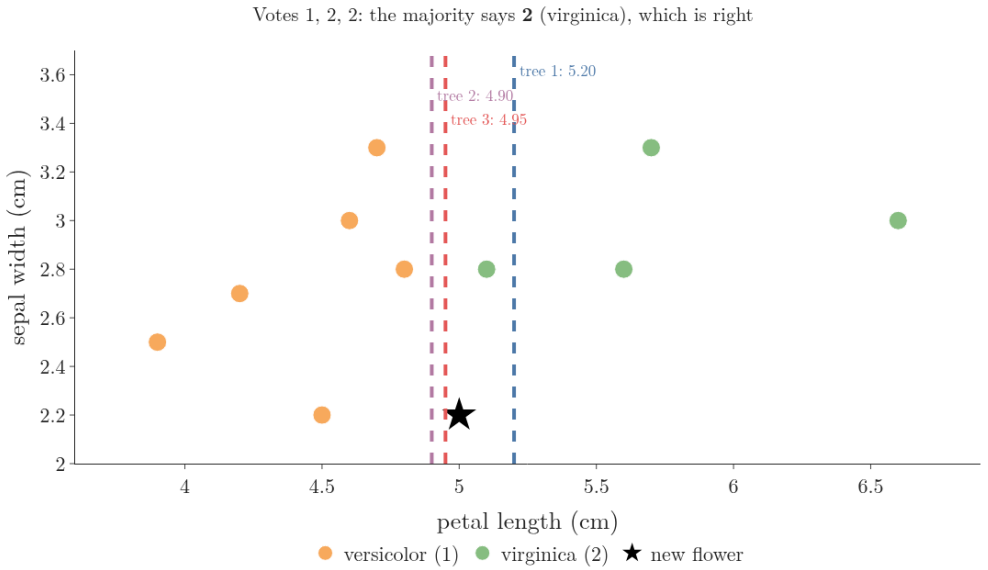


Figure 6: Bagging by hand. Each frame shows one tree's bootstrap sample (bigger dots are drawn more often, hollow dots are not drawn) and its cut on petal length; the last frame shows all three cuts and the vote on the new flower (star).

The majority says **2**, which is correct, although tree 1 alone would have been wrong. Each tree here saw only 8 observations, so it made just one split. With more observations, trees grow deeper: in column sampling and combined sampling, trees trained on 100 observations reach depths 3 to 6.

6. Types of bagging

> **Key point:** Sample observations with replacement (bagging) or without (pasting), sample features only (random subspaces), or both (random patches).

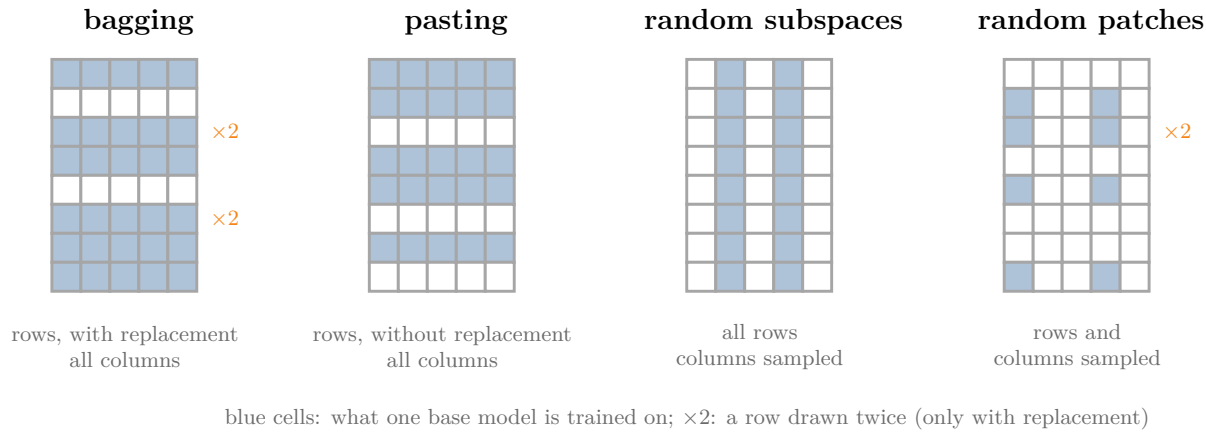


Figure 7: What one base model sees under each type: blue cells are the rows (observations) and columns (features) it is trained on

So far we sampled **observations** (rows of the table). We can sample **features** (columns) too: instead of giving each tree every feature, we give it a random subset. Feature sampling is meant for data with many features; pasting, random subspaces and random patches shows it hurting when there are only two. Combining these choices gives four types (Figure 7), each with its own name in the literature (scikit-learn User Guide, "Bagging meta-estimator").

6.1 Bagging

> **Key point:** Observation sampling with replacement.

What we have done so far: each model gets random observations, drawn with replacement, and all the features (Breiman, 1996).

6.2 Pasting

> **Key point:** Observation sampling without replacement: no observation repeats inside a sample.

Pasting (Breiman, 1999) is the same as bagging except that observations are drawn **without replacement**. In pandas, `df_train.sample(8)` gives 8 rows with no repeats, because `replace=False` is the default.

6.3 Random subspaces

> **Key point:** Feature sampling only: every model gets all the observations but only some of the features.

In **random subspaces** (Ho, 1998), there is no observation sampling. Each model gets every observation but a random subset of the features, drawn with or without replacement.

> **Python:** Sampling columns.
> `df1.sample(2, replace=True, axis=1)  # 2 random columns`
> `axis=1` tells `sample` to pick columns instead of rows. With `replace=True` the same column can be picked twice.

6.4 Random patches

> **Key point:** Observation sampling and feature sampling together.

In **random patches** (Louppe and Geurts, 2012), each model gets a random set of observations **and** a random set of features. In pandas we chain two samples:

> **Python:** Sampling rows, then columns.
> `df1.sample(8, replace=True).sample(2, replace=True, axis=1)`

All four aim at the same thing: give each base model different data, so the models differ. The bagging classifier runs all four in scikit-learn and compares them.

7. Summary

Type	Observations (rows)	Features (columns)
Bagging	sampled, with replacement	all
Pasting	sampled, without replacement	all
Random subspaces	all	sampled (with or without replacement)
Random patches	sampled	sampled

- All four types in the table aim at one thing: give each base model different data, so the models differ and their vote helps.
- Bagging = bootstrapping (each model trains on random observations) + aggregation (mode or mean of the predictions), because each bootstrap sample stands in for one fresh dataset, so each model is "what we would have learned from another dataset".
- All base models use the same algorithm; the variety comes from the data, so the voting argument still applies without mixing algorithms.
- A bootstrap sample holds about 63.2% of the distinct observations; the rest are out-of-bag, because drawing with replacement repeats some observations and misses others; those unseen observations can later score each model.
- Bagging keeps the low bias of a flexible model and cuts its **variance** (G-2073): 0.160 to 0.076 in Figure 4, because a change in the data is spread across many models, so no single change can swing the average.
- Use it whenever a model overfits (an unstable model); decision trees are the usual choice, because they are unstable and so gain the most (a 52 percent cut against 24 percent for 5-nearest neighbours), but any unstable algorithm can be bagged.
- So bagging gives what a single model rarely can: low **bias** (G-287) and low variance together.

8. Sources

Built from

- CampusX, "Bagging | Introduction | Part 1", YouTube, https://www.youtube.com/watch?v=LUiBOAy
- StatQuest with Josh Starmer, "Bootstrapping Main Ideas!!!", YouTube, https://www.youtube.com/wat 8-cgaQ (section 2.4)

Other references

- Breiman, L. (1996). "Bagging Predictors". *Machine Learning* 24(2), 123–140.
- Breiman, L. (1999). "Pasting Small Votes for Classification in Large Databases and On-Line". *Machine Learning* 36(1), 85–103.
- Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman and Hall.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning* (ESL), 2nd ed. Springer, sections 8.7 and 15.2.
- Ho, T. K. (1998). "The Random Subspace Method for Constructing Decision Forests". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20(8), 832–844.

- Louppe, G. and Geurts, P. (2012). "Ensembles on Random Patches". *Machine Learning and Knowledge Discovery in Databases* (ECML PKDD 2012), LNCS 7523, Springer, 346–361.
- scikit-learn developers. User Guide, "Bagging meta-estimator". scikit-learn.org, modules/ensemble.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Aggregation (in an ensemble) (G-183)	Combining the base models' predictions into one: mode for classes, mean for numbers.
High bias, low variance algorithm (G-895)	An algorithm too simple to fit the training data well but stable across samples, such as linear regression; it underfits.
Low bias, high variance algorithm (G-1132)	An algorithm that fits its training data very well but changes a lot with the data, such as a fully grown tree; it overfits.
With replacement (G-2125)	Sampling in which each drawn item is put back, so it can be drawn again.
Variance (of a model) (G-2073)	How much a model's predictions change when it is trained on a different sample of the data; a different meaning from the variance of a column.
Bagging regressor (G-252)	A bagging ensemble of regressors that predicts the mean of their predictions.
Standard error (G-1872)	How much a statistic, such as the sample mean, changes from one sample to the next, so how precise it is as an estimate. It is the standard deviation of the sampling distribution; for the mean, $\sigma/\sqrt{n}$.
Bias (of a model) (G-287)	Error from a model being too simple to capture the true relationship.
max_depth (G-1184)	The cap on a tree's depth; None lets it grow until every leaf is pure.
K-nearest neighbours (KNN) (G-998)	Predicting from the answers of the k closest stored points.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).

Term	Meaning
Target (G-1949)	The output column we predict, such as the class; also called the label.
Bootstrapping (G-322)	Drawing random samples of the data to train each base model.
Unstable model (G-2156)	A model whose fit changes a lot when the training data changes a little; bagging helps it most.
Out-of-bag observations (out-of-bag rows) (G-1412)	The observations a base model never saw because its bootstrap sample missed them (about 37%).
Pasting (G-1463)	Training several models, each on its own random sample of rows drawn without replacement (no row twice), and combining their predictions; it is bagging with this sample instead of a bootstrap sample.
Random subspaces (G-1618)	Training several models, each on all rows but a random subset of columns, and combining their predictions (a kind of bagging).
Random patches (G-1614)	Training several models, each on a random subset of rows and a random subset of columns, and combining their predictions (a kind of bagging).